

Mathematical Foundations for Symmetric Programming

Ohad Kammar, LFCS, University of Edinburgh
Matija Pretnar, FMF, University of Ljubljana



Shameless PLUG
(Programming Languages at University of Glasgow Seminar)
27 May 2026



THE UNIVERSITY OF EDINBURGH
informatics lfcs

Laboratory for Foundations
of Computer Science



supported by:

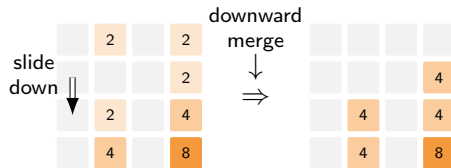
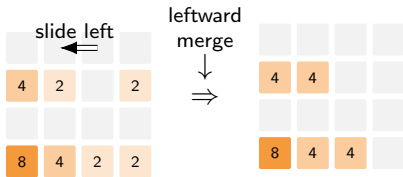


Advanced
Research
+ Invention
Agency

ARIA

Goal: exploit **symmetry** for **computation** and **reasoning**

Example: 2048 game¹ [Cirulli'14]



¹Available to play: <https://play2048.co>

2048 in Idris 2 [Brady'21]

Merging left goes with the grain

```
Row, Board : Nat -> Type
Row  n = Vect n Int
Board n = Vect n (Row n)

(.pad0) : Row n -> Row (1 + n)
xs.pad0 = replace {p = Row}
           (plusCommutative _ _)
           $ xs ++ [0]

mergeRow : Row n -> Row n
mergeRow [] = []
mergeRow ( 0 :: xs) = (mergeRow xs).pad0
mergeRow ( x :: []) = [x]
mergeRow (x :: 0 :: xs) = (mergeRow (x :: xs)).pad0
mergeRow (x :: y :: xs) with (x == y)
  _ | True  = 2*x :: (mergeRow xs).pad0
  _ | False =  x :: mergeRow (y :: xs)

mergeLeft : Board n -> Board n
mergeLeft = map mergeRow
```

Merging in 2048

Going against the grain

merging
up/down
incompatible
with inductive
structure



use a different
representation



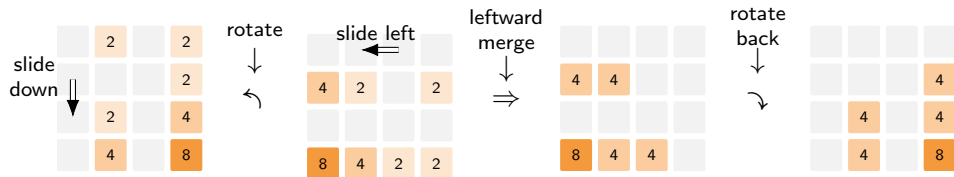
parameters
galore!²

```
var vector      = this.getVector(direction);
var traversals = this.buildTraversals(vector);
// ...
GameManager.prototype.getVector =
function (direction) { // Vectors representing tile movement
  var map = { 0: { x: 0, y: -1 }, // Up
              1: { x: 1, y: 0 },  // Right
              2: { x: 0, y: 1 },  // Down
              3: { x: -1, y: 0 } }; // Left
  return map[direction];
};
```

²See the function `GameManager.prototype.move`:

https://github.com/gabrielecirulli/2048/blob/478b6ec346e3787f589e4af751378d06ded4cbbc/js/game_manager.js#L129.

Easy 2048 implementation using **symmetry**

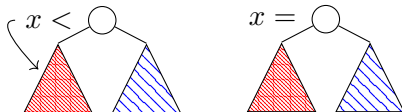


Anecdotal evidence with half-a-dozen UG students

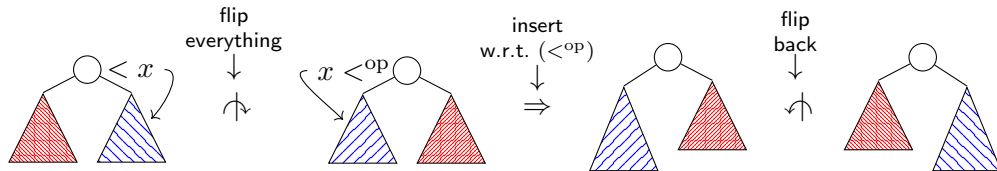
- ✗ buggy parametric solution
- ✓ mostly right symmetric solution

Example: binary search tree insertion using **symmetry**

Interesting cases



Symmetric case



NB: Yes, silly performance. Won't improve performance today, more later.

Example: **proofs** using **symmetry** [adapted from Harrison'09]

Theorem (Schur's inequality)

Then, for all $x_1, x_2, x_3 > 0$, we have, for all t :

$$x_1^t(x_1 - x_2)(x_1 - x_3) + x_2^t(x_2 - x_3)(x_2 - x_1) + x_3^t(x_3 - x_1)(x_3 - x_2) \geq 0$$

Proof

Without loss of generality, assume $x_1 \geq x_2 \geq x_3$. Then:

$$\begin{aligned} x_1^t(x_1 - x_2)(x_1 - x_3) + x_2^t(x_2 - x_3)(x_2 - x_1) + x_3^t(x_3 - x_1)(x_3 - x_2) &= \\ (x_1 - x_2) \left(x_1^t(x_1 - x_3) - x_2^t(x_2 - x_3) \right) + x_3^t(x_1 - x_3)(x_2 - x_3) &\geq \\ (x_1 - x_2)(x_1^t - x_2^t)(x_1 - x_3) + x_3^t(x_1 - x_3)(x_2 - x_3) &\geq 0 \end{aligned}$$



Example: **proofs** using **symmetry** [adapted from Harrison'09]

Theorem (Schur's inequality)

Then, for all $x_1, x_2, x_3 > 0$, we have, for all t :

$$x_1^t(x_1 - x_2)(x_1 - x_3) + x_2^t(x_2 - x_3)(x_2 - x_1) + x_3^t(x_3 - x_1)(x_3 - x_2) \geq 0$$

Proof

Without loss of generality, assume $x_1 \geq x_2 \geq x_3$. Then:

$$\begin{aligned} x_1^t(x_1 - x_2)(x_1 - x_3) + x_2^t(x_2 - x_3)(x_2 - x_1) + x_3^t(x_3 - x_1)(x_3 - x_2) &= \\ (x_1 - x_2) \left(x_1^t(x_1 - x_3) - x_2^t(x_2 - x_3) \right) + x_3^t(x_1 - x_3)(x_2 - x_3) &\geq \\ (x_1 - x_2)(x_1^t - x_2^t)(x_1 - x_3) + x_3^t(x_1 - x_3)(x_2 - x_3) &\geq 0 \end{aligned}$$



Rocq SSReflect³

The general syntax of without loss is:

```
wlog suff? clear_switch? i_item? : ident* / term
```

On a goal **G**, it creates two subgoals: a first one to prove the formula $(\text{term} \rightarrow \mathbf{G}) \rightarrow \mathbf{G}$ and a second one to prove the formula $\text{term} \rightarrow \mathbf{G}$. If the optional list of `ident` is present on the left side of `/`, these constants are generalized in the premise $(\text{term} \rightarrow \mathbf{G})$ of the first subgoal.

³Rocq Reference Manual:

<https://rocq-prover.org/doc/V9.2.0/refman/proof-engine/ssreflect-proof-language.html#rocq:tacn.wlog>

Agda⁴

```
-- Without Loss of Generality
```

```
module _ {_R_ : Rel A ℓ1} {Q : Rel A ℓ2} where
```

```
wlog : Total _R_ → Symmetric Q →  
      (∀ a b → a R b → Q a b) →  
      ∀ a b → Q a b
```

⁴Agda Stdlib:

<https://github.com/agda/agda-stdlib/blob/c2abce1b8e72ccae49f73791d0996eaf23674357/src/Relation/Binary/Consequences.agda#L197>

Lean4⁵

The tactic `wlog h : P` will add an assumption `h : P` to the main goal, and add a new goal that requires showing that the case $\neg h : P$ can be reduced to the case where `P` holds (typically by symmetry).

⁵Mathlib4 documentation:

https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/WLOG.html

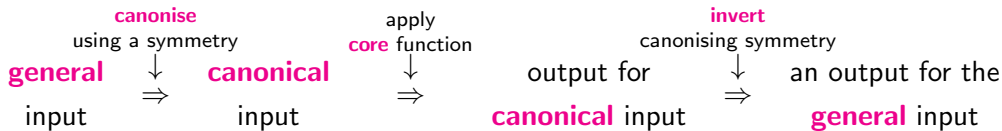
uniform w.l.o.g. using symmetry

$$\begin{array}{ccccccc} & \text{sort using } \pi & & \text{prove using} & & \text{permute back} & \\ & \downarrow & & \downarrow & & \downarrow & \\ x_1, x_2, x_3 \in \mathbb{R}_+ & \Rightarrow & x_{\pi 1} \geq x_{\pi 2} \geq x_{\pi 3} & \Rightarrow & P(x_{\pi 1}, x_{\pi 2}, x_{\pi 3}) & \Rightarrow & P(x_1, x_2, x_3) \\ & & \text{additional assumption} & & \text{since } P \text{ is invariant} & & \end{array}$$

NB: revisionism:

- ✓ Dijkstra'95 [EWD1223, not mechanised]
- ✓ HOL WLOG tactics [Harrison'09]
- ✗ Rocq SSReflect
- ✓ Agda Stdlib
- ✗ Lean

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** function/proof)



symmetry = **group** + **action**

Symmetric programming

Exploit symmetries for computation and reasoning—**avoid symmetric cases**.

Mathematical foundations for symmetric programming semantics:

- ▶ **Abstractions** for symmetric programming
- ▶ **Simply-typed** reasoning about equivariance
- ▶ **Dependently-typed** and mathematical reasoning
- ▶ User-defined **data-types** through initial algebra semantics and induction principles
 - ▶ Simply-typed data-types
 - ▶ Dependently-typed/indexed data-types

Talk structure

- ▶ Introduction
- ▶ Remedial group theory basics
- ▶ Simply-typed symmetric programming
- ▶ Dependently-typed symmetric programming
 - ▶ Indexed actions and w.l.o.g.
 - ▶ Mathematical reasoning: Dijkstra'95 & Harrison'09
- ▶ Symmetric user-defined data-types
 - ▶ Simply-typed data-types
 - ▶ Indexed data-types
- ▶ Not today: Equivariant reasoning
- ▶ Conclusion

Remedial group theory basics

Group action $\langle G, A \rangle$

Group $G = \langle \underline{G}, \dots \rangle$

G -action $A = \langle \underline{A}, \dots \rangle$

Intuition

- ▶ **symmetries**: $\underline{G}$ elements
- ▶ x **symmetric to** y **via** g

action and group axioms mean:

‘**being symmetric**’ is a
proof-relevant equivalence relation tracked by
symmetries

Remedial group theory basics

Group action $\langle G, A \rangle$

Group $G = \langle \underline{G}, (*), e, (-)^{-1} \rangle$:

$\underline{G}$ (**carrier** set) $\frac{g, h : \underline{G}}{g * h : \underline{G}}$ (**multiplication/composition**)

$\frac{}{e : \underline{G}}$ (**unit/identity/neutral**) $\frac{g : \underline{G}}{g^{-1} : \underline{G}}$ (**inverse**)

such that:

$g, h, k : \underline{G} \vdash (g * h) * k = g * (h * k)$ (associativity)

$g : \underline{G} \vdash g * e = g = e * g$ (neutrality)

$g : \underline{G} \vdash g * g^{-1} = e = g^{-1} * g$ (inverses)

G-action $A = \langle \underline{A}, \dots \rangle \dots$

Board rotations $\langle \mathbf{C}_4, \text{Dir} \rangle$

$\underline{\mathbf{C}}_4 := \{\hookleftarrow, \curvearrowright, \curvearrowleft, \mathbf{Q}\}$

$\hookleftarrow * \hookleftarrow := \curvearrowright \quad \hookleftarrow * \curvearrowright := \curvearrowleft$

$\hookleftarrow * \curvearrowleft := \mathbf{Q} \quad \hookleftarrow * \mathbf{Q} := \hookleftarrow$

group laws imply rest and:

$e = \mathbf{Q}$

$\hookleftarrow^{-1} = \curvearrowleft \quad \curvearrowright^{-1} = \curvearrowright$

Remedial group theory basics

Group action $\langle G, A \rangle$

Group $G = \langle \underline{G}, (*), e, (-)^{-1} \rangle$

G-action $A = \langle \underline{A}, \overset{A}{*} \rangle :$

$$\underline{A}(\text{carrier set}) \quad \frac{g : \underline{G} \quad x : \underline{A}}{g \overset{A}{*} x : \underline{A}} (\text{action of } G \text{ on } \underline{A})$$

such that:

$$\begin{aligned} g, h : \underline{G}, x : \underline{A} \vdash g \overset{A}{*} (h \overset{A}{*} x) &= (g * h) \overset{A}{*} x && \text{(associativity)} \\ x : \underline{A} \vdash e \overset{A}{*} x &= x && \text{(neutrality)} \end{aligned}$$

Board rotations $\langle \mathbf{C}_4, \text{Dir} \rangle$

$$\underline{\mathbf{C}}_4 := \{ \swarrow, \searrow, \nearrow, \nwarrow \}$$

$$\begin{aligned} \swarrow * \swarrow &:= \searrow & \swarrow * \searrow &:= \nearrow \\ \swarrow * \nearrow &:= \nwarrow & \swarrow * \nwarrow &:= \swarrow \end{aligned}$$

acting on

$$\underline{\text{Dir}} := \{ \Leftarrow, \Rightarrow, \Uparrow, \Downarrow \}$$

$$\begin{aligned} \swarrow * \Leftarrow &:= \Downarrow & \swarrow * \Downarrow &:= \Rightarrow \\ \swarrow * \Rightarrow &:= \Uparrow & \swarrow * \Uparrow &:= \Leftarrow \end{aligned}$$

laws determine the rest
acting similarly on:

$$\text{Board}_n \text{ and } \text{Dir} \times \text{Board}_n$$

Remedial group theory basics

Group action $\langle G, A \rangle$

Group $G = \langle \underline{G}, (*), e, (-)^{-1} \rangle$

G-action $A = \langle \underline{A}, \rangle$

Notation

► **symmetries**: $\underline{G}$ elements

► x **symmetric to** y **via** g when: $x = g \circledast y$

action and group axioms mean:

‘**being symmetric**’ is a
proof-relevant equivalence relation tracked by
symmetries

Board rotations $\langle \mathbf{C}_4, \mathbf{Dir} \rangle$

$\underline{\mathbf{C}}_4 := \{ \curvearrowleft, \curvearrowright, \curvearrowup, \curvearrowdown \}$

$\curvearrowleft * \curvearrowleft := \curvearrowright \quad \curvearrowleft * \curvearrowright := \curvearrowup$

$\curvearrowleft * \curvearrowup := \curvearrowdown \quad \curvearrowleft * \curvearrowdown := \curvearrowleft$

acting on

$\underline{\mathbf{Dir}} := \{ \Leftarrow, \Rightarrow, \Uparrow, \Downarrow \}$

$\curvearrowleft \circledast \Leftarrow := \Downarrow \quad \curvearrowleft \circledast \Downarrow := \Rightarrow$

$\curvearrowleft \circledast \Rightarrow := \Uparrow \quad \curvearrowleft \circledast \Uparrow := \Leftarrow$

laws determine the rest
acting similarly on:

$\mathbf{Board}_n$ and $\mathbf{Dir} \times \mathbf{Board}_n$

Symmetry-preserving functions

Equivariant map $\varphi : A \circledast \rightarrow B$ between G -actions A, B

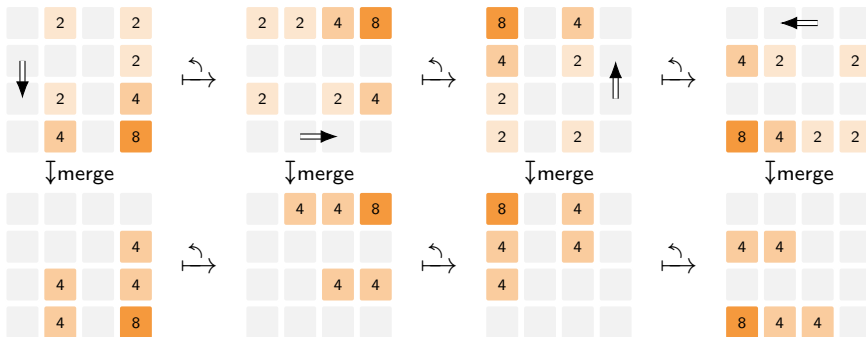
$\varphi : \underline{A} \rightarrow \underline{B}$
function

$g : \underline{G}, x : \underline{A} \vdash$

$\varphi(g \circledast x) = g \circledast (\varphi x) : \underline{B}$

i.e.: $\frac{x, y \text{ symmetric via } g}{f x, f y \text{ symmetric via } g}$

Example: $\text{merge} : \text{Dir} \times \text{Board}_n \circledast \rightarrow \text{Board}_n$



Talk structure

- ▶ Introduction
- ▶ Remedial group theory basics
- ▶ **Simply-typed symmetric programming**
- ▶ Dependently-typed symmetric programming
 - ▶ Indexed actions and w.l.o.g.
 - ▶ Mathematical reasoning: Dijkstra'95 & Harrison'09
- ▶ Symmetric user-defined data-types
 - ▶ Simply-typed data-types
 - ▶ Indexed data-types
- ▶ Not today: Equivariant reasoning
- ▶ Conclusion

w.l.o.g. anatomy: **canonisation**

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** function/proof)

Canoniser for G -action A

function $c : \underline{A} \rightarrow \underline{G}$ assigning **canonising symmetries** to A -elements.

Canonical elements

$$\text{Can } c := \{ (c a) \otimes a \mid a \in \underline{A} \}$$

Example: $\text{mkLft} : \underline{\text{Dir}} \rightarrow \underline{\text{C}}_4$

Canonising symmetry makes direction 'left':

$$\text{mkLft} \Rightarrow := \curvearrowleft$$

$$\text{mkLft} \Uparrow := \curvearrowright$$

$$\text{mkLft} \Leftarrow := \mathbb{Q}$$

$$\text{mkLft} \Downarrow := \curvearrowleft$$

Only canonical direction is 'left':

$$\text{Can } \text{mkLft} = \{ \Leftarrow \}$$

Lemma

$$\text{Can} \left(\underline{A} \times \underline{B} \xrightarrow{\pi_1} \underline{A} \xrightarrow{c} \underline{G} \right) = \text{Can} \left(\underline{A} \xrightarrow{c} \underline{G} \right) \times \underline{B}$$

Example

Canonise merge-inputs by canonising the direction:

$$c : \underline{\text{Dir}} \times \underline{\text{Board}}_n \xrightarrow{\pi_1} \underline{\text{Dir}} \xrightarrow{\text{mkLft}} \underline{\mathbf{C}}_4$$

By this Lemma, in canonical merge-inputs, merge-direction left but board unrestricted:

$$\text{Can } c = \{ \Leftarrow \} \times \underline{\text{Board}}_n$$

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** function/proof)

Core function $f : \text{Can } c \rightarrow \underline{B}$

work on canonical elements, typically more concrete, after eliminating symmetric cases

Example: merging left

$\text{merge}_{\leftarrow}^{\text{row}} : \text{Row}_n \rightarrow \text{Board}_n$

$\text{merge}_{\leftarrow}^{\text{row}} \langle \rangle := \langle \rangle$

$\text{merge}_{\leftarrow}^{\text{row}} \langle 0 \rangle \mathbin{++} r := (\text{merge}_{\leftarrow}^{\text{row}} r) \mathbin{++} \langle 0 \rangle$

$\text{merge}_{\leftarrow}^{\text{row}} \langle x \rangle := \langle x \rangle$

$\text{merge}_{\leftarrow}^{\text{row}} (\langle x, 0 \rangle \mathbin{++} r) := \text{merge}_{\leftarrow} (\langle x \rangle \mathbin{++} r) \mathbin{++} \langle 0 \rangle$

$\text{merge}_{\leftarrow}^{\text{row}} (\langle 2^k, 2^k \rangle \mathbin{++} r) := \langle 2^{k+1} \rangle \mathbin{++} (\text{merge}_{\leftarrow}^{\text{row}} r) \mathbin{++} \langle 0 \rangle$

$\text{merge}_{\leftarrow}^{\text{row}} (\langle 2^k, 2^\ell \rangle \mathbin{++} r) := \langle 2^k \rangle \mathbin{++} \text{merge}_{\leftarrow}^{\text{row}} (\langle 2^\ell \rangle \mathbin{++} r)$

$\text{merge}_{\leftarrow} : \text{Board}_n \rightarrow \text{Board}_n$

$\text{merge}_{\leftarrow} := \text{map } \text{merge}_{\leftarrow}^{\text{row}}$

Symmetric programming **abstraction**

w.l.o.g. = (input & output **symmetries**)+(**canonising** symmetries)+(**core** function/proof)

W.l.o.g. construction

Formally:

$$\frac{G\text{-actions } A, B \quad c : \underline{A} \rightarrow \underline{G} \quad f : \text{Can } c \rightarrow \underline{B}}{\text{wlog}(c, f) := \left(a \mapsto (ca)^{-1} \circledast f((ca) \circledast a) \right) : \underline{A} \rightarrow \underline{B}}$$

Example

```
mergeUnchecked : {n : Nat} -> Dir -> Board n -> Board n
```

```
mergeUnchecked = Prelude.curry $ wlog
```

```
  (\(dir, _) => makeLeft dir)           -- canoniser
  (\case (Lt, board) => mergeLeft board  -- core function
    _ => believe_me "oops")             -- unreachable case
```

Symmetric programming **abstraction**

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** function/proof)

W.l.o.g. construction

Formally:

$$\frac{G\text{-actions } A, B \quad c : \underline{A} \rightarrow \underline{G} \quad f : \text{Can } c \rightarrow \underline{B}}{\text{wlog}(c, f) := \left(a \mapsto (ca)^{-1} \otimes f((ca) \otimes a) \right) : \underline{A} \rightarrow \underline{B}}$$

Example

```
core : {n : Nat} -> Canon (Canoniser {n}) -> Board n
```

```
core ((d, b) ** ev) with 0 (canonicalView ev)  
  core ((Lt, b) ** ev) | Refl = mergeLeft b
```

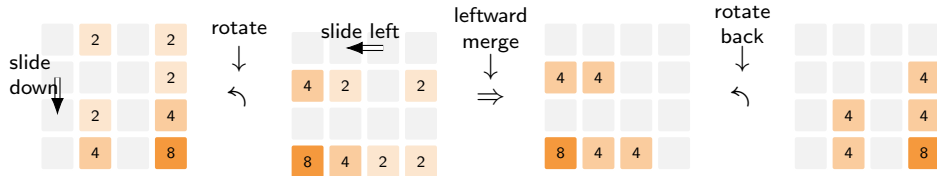
```
merge : {n : Nat} -> Dir -> Board n -> Board n
```

```
merge = Prelude.curry $ wlog' Canoniser core
```

Symmetric programming in Idris 2 [Brady'21]

Simply-typed w.l.o.g.

- ▶ Straightforward to implement
- ▶ Straightforward to use
- ▶ Plumbing group actions with type-classes/interfaces fiddly as usual
- ▶ Plenty of 'believe_me' in code for unreachable cases
- ▶ Doesn't cover mathematical reasoning



TwentyFortyEight.idr

Talk structure

- ▶ Introduction
- ▶ Remedial group theory basics
- ▶ Simply-typed symmetric programming
- ▶ **Dependently-typed symmetric programming**
 - ▶ Indexed actions and w.l.o.g.
 - ▶ Mathematical reasoning: Dijkstra'95 & Harrison'09
- ▶ Symmetric user-defined data-types
 - ▶ Simply-typed data-types
 - ▶ Indexed data-types
- ▶ Not today: Equivariant reasoning
- ▶ Conclusion

Back to mathematics

these concepts apply to **propositions** & **proofs**

concepts apply **uniformly** via dependent types:

types as propositions; programs as proofs



G-actions on propositions; **canonisers** and **core functions** in **wlog** proofs

easy with **simple types** (HOL/FOL), **duplicating** concepts in different **kinds/stages**
(terms vs. propositions, object-level vs. meta-level)

analyse extant w.l.o.g. principles of Dijkstra & Harrison

Digression: mathematics with dependent types

keep meta-language **implicit**:

mathematical foundations have no room in polite mathematical discourse

instead, use concepts that **explicate** type dependency:

indexed families and **terms-in-context**

analogy: vernacular use of sets, functions, relations, etc.

without necessarily committing to ZF, nor requiring moving to a meta-level

one can mould families into **various models** of **various type theories**:

I don't want to, it adds **little value** and **much noise** at the mathematical level

formalisation and esp. **mechanisation** add real value,

but cement a specific (formal) meta-language—left for **future work**

Explicating type dependency

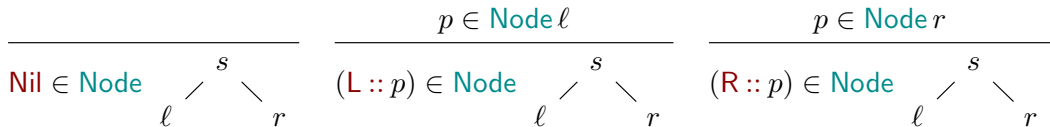
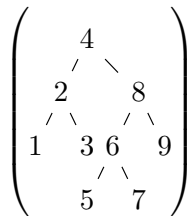
Family $\Gamma \vdash X$

- ▶ Γ set of **indices**
- ▶ X a family assigning a set X_γ to each index $\gamma \in \Gamma$ —the **fibre** over γ

Example: internal tree nodes $t : \text{Tree } S \vdash \text{Node } t$

- ▶ index $\text{Tree } S \ni t$ binary tree with labels in S ; $R :: L :: \text{Nil} \in \text{Node}$
- ▶ fibre $\text{Node } t$: paths ending in internal node of $\text{Tree } S \ni t$.

Fibres collectively form the smallest sets of lists over $\{L, R\}$ containing:

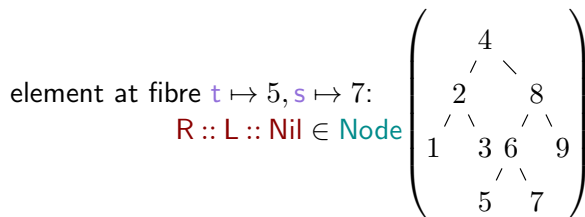
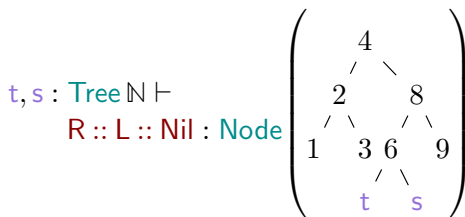


Explicating type dependency

Term/fibred element $\Gamma \vdash M : X$

- ▶ family $\Gamma \vdash X$
- ▶ M a sequence assigning an element $M_\gamma \in X_\gamma$ at every fibre over $\gamma \in \Gamma$

Example



Formalising type dependency

Straightforward⁶ abstractions for type dependency:

$$\begin{array}{c}
 \frac{\Gamma \vdash X}{\Gamma, x : X \text{ set}} \quad \frac{\theta : \Gamma \rightarrow \Delta \quad \Delta \vdash Y}{\Gamma \vdash Y [\theta]} \quad \frac{\Gamma \vdash X}{\text{wkn} : (\Gamma, x : X) \rightarrow \Gamma} \quad \frac{\Gamma \vdash X}{\Gamma, x : X \vdash x : X [\text{wkn}]} \\
 \\
 \frac{\theta : \Delta \rightarrow \Gamma \quad \Delta \vdash M : X [\theta]}{(\theta, x \mapsto M) : \Delta \rightarrow (\Gamma, x : X)} \quad \frac{\Gamma \vdash X \quad \Gamma, x : X \vdash Y}{\Gamma \vdash (x : X) \times Y} \quad \frac{\Gamma \vdash M : X \quad \Gamma \vdash K : Y [x \mapsto M]}{\Gamma \vdash M, K : (x : X) \times Y} \\
 \\
 \frac{\Gamma \vdash M : (x : X) \times Y}{\Gamma \vdash M.\text{fst} : X} \quad \frac{\Gamma \vdash M : (x : X) \times Y}{\Gamma \vdash M.\text{snd} : Y [x \mapsto M.\text{fst}]} \quad \frac{\Gamma \vdash X \quad \Gamma, x : X \vdash Y}{\Gamma \vdash (X : x) \rightarrow Y} \\
 \\
 \frac{\Gamma \vdash K : (X : x) \rightarrow Y \quad \Gamma \vdash M : X}{\Gamma \vdash K M : Y [x \mapsto M]} \quad \frac{\Gamma, x : X \vdash M : Y}{\Gamma \vdash (\lambda x : X. M) : (x : X) \rightarrow Y}
 \end{array}$$

⁶By 'straightforward', I mean: experienced type theorists 'do the usual thing'. Inexperienced theorists need to learn 'the usual things', so working it out may take some time and guidance.

Formalising type dependency

Straightforward⁷ because defer type-dependency difficulties to meta-level:

- ▶ Dependent-pairs/Grothendieck construction $\coprod_{\Gamma} X := \{\gamma, x \mid \gamma \in \Gamma, x \in X_{\gamma}\}$
- ▶ Dependent-functions/products $\prod_{\Gamma} X$

Can turn these into a **formal** or even **mechanised** type theory, and mould families into your **favourite model** of this type-theory, including a **shallow embedding** if your metalanguage is dependently-typed.

As usual, your mileage may vary, and independently of this development.
(So that's on you, and we can discuss offline.)

End of digression, returning to **dependently-typed symmetric programming**

⁷By 'straightforward', I mean: experienced type theorists 'do the usual thing'. Inexperienced theorists need to learn 'the usual things', so working it out may take some time and guidance.

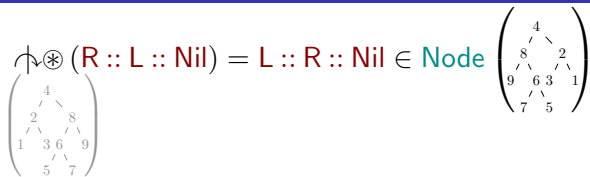
Dependently-typed actions

Indexed G -action $\Gamma \vdash A$

► $\Gamma = \langle \underline{\Gamma}, (\otimes) \rangle$ **indexing** G -action

► $\underline{\Gamma} \vdash \underline{A}$ **carrier** family

► $\frac{\vdash g : \underline{G} \quad \gamma : \Gamma \vdash a : \underline{A}}{\gamma : \Gamma \vdash g \otimes_{\gamma} a : \underline{A} [\gamma \mapsto g \otimes \gamma]} \text{ } \Gamma\text{-indexed action of } G \text{ on } \underline{A}$



such that

equivalently:

$$(\otimes) : (g : \underline{G}) \rightarrow (\gamma : \Gamma) \rightarrow \underline{A}_{\gamma} \rightarrow \underline{A}_{g \otimes \gamma}$$

$$\gamma : \Gamma, x : \underline{A} \vdash e \otimes_{\gamma} x = x \quad g, k : \underline{G}, \gamma : \Gamma, x : \underline{A} \vdash (g * k) \otimes_{\gamma} x = g \otimes_{k \otimes \gamma} (k \otimes_{\gamma} x)$$

Proposition

Indexed actions amount to equivariant projections from the Grothendieck construction:

$$\begin{array}{ccc} \Gamma \vdash A & \iff & \coprod_{\Gamma} A = (\coprod_{\underline{\Gamma}} \underline{A}, ((\otimes), (\otimes))) \\ \text{indexed } G\text{-action} & & G\text{-action} \end{array} \quad \pi_1 : \coprod_{\Gamma} A \multimap \Gamma$$

Indexed w.l.o.g. anatomy: canonisers

Indexed canoniser for G -action $\Gamma \vdash A$

term $\underline{\Gamma} \vdash c : \underline{A} \rightarrow \underline{G}$

induced **family** $\underline{\Gamma} \vdash \text{Can } c$ of canonical elements:

$$(\text{Can } c)_\gamma := \left\{ g \circledast a \middle| g \in \underline{G}, a \in \underline{A}_{g^{-1} \circledast \gamma}, c_{g^{-1} \circledast \gamma} a = g \right\}$$

Example

A canoniser for paths indexed by binary trees with action of $\mathbf{S}_2 := \{\mathbf{I}, \swarrow\}$:

$$t : \text{Tree } \mathbb{N} \vdash c : \text{Node } t \rightarrow \mathbf{S}_2 \quad c_{\text{Node}\langle \ell, n, r \rangle} \text{Nil} := \mathbf{I} =: c_{\text{Node}\langle \ell, n, r \rangle} \mathbf{L} \quad c_{\text{Node}\langle \ell, n, r \rangle} \mathbf{R} := \swarrow$$

Canonical paths may only turn **R** after first step:

$$(\text{Can } c)_{\text{Node}\langle \ell, n, r \rangle} = \{\text{Nil}\} \cup \{\mathbf{L} :: p \mid p \in \text{Node } \ell\}$$

Indexed w.l.o.g. anatomy: core functions

Indexed core function

term $\underline{\Gamma} \vdash f : (\mathbf{x} : \text{Can } c) \rightarrow \underline{B}_{\text{coerce}_{\text{Can } c \subseteq \underline{A}}} \mathbf{x}$

Example

core function returning node pointed at by a path:

$$\begin{aligned} \mathbf{t} : \text{Tree } \mathbb{N} \vdash \text{lookup}^{\text{Core}} : \text{Can } c \rightarrow \mathbb{N} \quad \text{lookup}^{\text{Core}}_{\text{Node}\langle \ell, m, r \rangle} \text{Nil} &:= m \\ \text{lookup}^{\text{Core}}_{\text{Node}\langle \ell, m, r \rangle} (\mathbf{L} :: p) &:= \text{lookup}_{\ell} p \end{aligned}$$

- ✓ **eliminated** symmetric case
- ✗ **recursive** call to lookup, not lookup^{Core}
revisit through **data-types**

Symmetric programming **dependently-typed** abstraction

indexed w.l.o.g. = (**indexed** symmetries) + (**indexed** canoniser) + (**indexed** core function)

W.l.o.g. construction

Formally:

$$\frac{G\text{-actions } \Gamma \vdash A \quad \Gamma, x : A \vdash B \quad \Gamma \vdash c : \underline{A} \rightarrow \underline{G} \quad \Gamma \vdash f : \text{Can } c \rightarrow \underline{B}}{\gamma : \underline{\Gamma} \vdash \text{wlog}(c, f) := \left(\lambda a. (ca)^{-1} \underset{(ca) \circledast \gamma}{\circledast} f \left((ca) \underset{\gamma}{\circledast} a \right) \right) : (x : \underline{A}) \rightarrow \underline{B}}$$

Example (revisit later)

$$t : \text{Tree } \mathbb{N} \vdash \text{lookup} := \text{wlog}(c, \text{lookup}^{\text{Core}}) : \text{Node } t \rightarrow \mathbb{N}$$

Talk structure

- ▶ Introduction
- ▶ Remedial group theory basics
- ▶ Simply-typed symmetric programming
- ▶ **Dependently-typed symmetric programming**
 - ▶ Indexed actions and w.l.o.g.
 - ▶ **Mathematical reasoning: Dijkstra'95 & Harrison'09**
- ▶ Symmetric user-defined data-types
 - ▶ Simply-typed data-types
 - ▶ Indexed data-types
- ▶ Not today: Equivariant reasoning
- ▶ Conclusion

Mathematical reasoning w.l.o.g.

Propositions as types, proofs as programs

For this part, take propositions $\varphi \in \mathbf{Prop} := \{\mathbf{True}, \mathbf{False}\}$. Use trivial action as needed.

$\mathbf{Prop}$ acts as a universe with:

$$\frac{\Gamma \vdash \varphi : \mathbf{Prop}}{\Gamma \vdash \varphi} \quad \varphi_\gamma := \begin{cases} \varphi_\gamma = \mathbf{True} : & \mathbb{1} \\ \text{otherwise} : & \emptyset \end{cases}$$

$$\begin{aligned} \text{E.g.: } (x, y, z) : (0, \infty)^3 \vdash \text{Schur} := \\ \forall t : \mathbb{R}. x_1^t (x_1 - x_2)(x_1 - x_3) \\ + x_2^t (x_2 - x_3)(x_2 - x_1) \\ + x_3^t (x_3 - x_1)(x_3 - x_2) \\ \geq 0 : \mathbf{Prop} \end{aligned}$$

At most one term $\Gamma \vdash M : \varphi$. It exists iff φ holds in every fibre $\gamma \in \Gamma$.

Equivariant terms $\Gamma \vdash M : A$ (where $\Gamma \vdash A$ indexed G -action)

terms $\underline{\Gamma} \vdash M : \underline{M}$ satisfying: $\underline{g} : \underline{G}, \gamma : \underline{\Gamma} \vdash g \underset{\gamma}{\circledast} M_\gamma = M_{\underline{g} \circledast \gamma} : A_{\underline{g} \circledast \gamma}$

Proposition

$\Gamma \vdash \varphi : \mathbf{Prop}$ **invariant**
i.e., equivariant w.r.t. trivial action,



$\Gamma \vdash \varphi$ has an indexed action
(this indexed action is unique)

Mathematical reasoning w.l.o.g.

Revisionism

Few sources treat w.l.o.g. formally, we don't encompass all extant uses

Use our w.l.o.g. principle for **taxonomy**:

- ✓ Dijkstra'95 [EWD1223, not mechanised]
- ✓ HOL WLOG tactics [Harrison'09]
- ✗ Rocq SSReflect
- ✓ Agda Stdlib
- ✗ Lean

Harrison's w.l.o.g. tactics

Harrison's position

Is w.l.o.g. an independent principle or a shorthand?

Implications for **formalisation**:

- ▶ w.l.o.g. as a **shorthand: reconstruct** omission
Harrison posits this view as: ugly/clumsy, inefficient, unfaithful
- ▶ w.l.o.g. **principle: capture** argument structure

Harrison posits a principle is more faithful, and formalises several specialised w.l.o.g. principles, e.g.⁸ by proving all six cases:

$$\frac{\begin{array}{l} \forall x, y, z : (0, \infty). P(x, y, z) \Rightarrow P(y, x, z) \wedge P(x, z, y) \\ \forall x, y, z : (0, \infty). x \leq y \wedge y \leq z \Rightarrow P(x, y, z) \end{array}}{\forall x, y, z : (0, \infty). P(x, y, z)} \quad (\text{wlog}_3)$$

⁸Available here: <https://github.com/jrh13/hol-light/blob/master/int.ml#L612> .

Harrison's w.l.o.g. tactics with symmetric programming

Splitting hairs

Harrison's ($wlog_3$) proof amounts to **shorthand**:

repeats the abstract proof for P six times for each input permutation.

Symmetric programming confines combinatorics to the permutation group S_3

Generating set $S \subseteq \underline{G}$

E.g.: $\{1 \leftrightarrow 2, 2 \leftrightarrow 3\}$ generates S_3

when: $\forall H \subseteq G$ sub-group. $S \subseteq \underline{H} \implies \underline{H} = \underline{G}$

Proposition (S generate G , indexed G -action Γ , proposition $\Gamma \vdash \varphi : \text{Prop}$)

$$\frac{g : S, \gamma : \Gamma \vdash \varphi [\gamma \mapsto g \circ \gamma] \iff \varphi}{\Gamma \vdash \varphi : \text{Prop is equivariant}}$$

Harrison's w.l.o.g. tactics with symmetric programming

$$\frac{\begin{array}{l} (1) \quad \forall x, y, z : (0, \infty). P(x, y, z) \Rightarrow P(y, x, z) \wedge P(x, z, y) \\ (2) \quad \forall x, y, z : (0, \infty). x \leq y \wedge y \leq z \Rightarrow P(x, y, z) \end{array}}{\forall x, y, z : (0, \infty). P(x, y, z)} \quad (\text{wlog}_3)$$

- ▶ $\Gamma := (x, y, z) : (0, \infty)^3$
- ▶ Premise (1) is premise in Proposition $\Rightarrow \gamma : \Gamma \vdash P\gamma : \text{Prop}$ equivariant
 $\Rightarrow$ indexed action of $\mathbf{S}_3$ on $\gamma : \Gamma \vdash P\gamma$
- ▶ Premise (2) is core function $p : (\gamma : \text{Can } c) \rightarrow P\gamma$ for sorting canoniser $c : \underline{\Gamma} \rightarrow \underline{\mathbf{S}}_3$
- ▶ Combinatorics-free proof: $\text{wlog}_3 := \text{wlog}(c, p)(x, y, z)$

Proposition (S generate G , indexed G -action Γ , proposition $\Gamma \vdash \varphi : \text{Prop}$)

$$\frac{g : S, \gamma : \Gamma \vdash \varphi [\gamma \mapsto g \circ \gamma] \Leftarrow \varphi}{\Gamma \vdash \varphi : \text{Prop is equivariant}}$$

Harrison's w.l.o.g. tactics with symmetric programming

Harrison's w.l.o.g. tactics are $wlog(c, p)$

by similar analysis, for suitable canoniser c and core function f

One exception⁹

$$\frac{\forall(\mathbf{x}, \mathbf{y}) : \mathbb{R}^2. P(\mathbf{x}, \mathbf{y}, 0)}{\forall(\mathbf{x}, \mathbf{y}, \mathbf{z}) : \mathbb{R}^3. \mathbf{z} = 0 \implies P(\mathbf{x}, \mathbf{y}, \mathbf{z})} \quad (\text{pad2d3d_tac})$$

(This presentation de-emphasises importance for automated goal-simplification.)

Takeaway

Revisionist wlog as crisp taxonomy for extant w.l.o.g. arguments

⁹Available here: <https://github.com/jrh13/hol-light/blob/master/int.ml#L612> .

Harrison's concluding remarks

Harrison connects his w.l.o.g. principles to symmetry, but not to group actions.

Harrison considers **domain-specific reflective libraries** for w.l.o.g. reasoning, such as ones for geometric reasoning. He argues they are **too specific**.

Groups actions maintain a flexible level of generality, and high expressivity.

Dijkstra's w.l.o.g. misery

Note EWD1223

presents a general w.l.o.g. principle for binary relations, with two goals:

- ▶ explicate it formally
- ▶ kvetch—using unordered pairs—about how w.l.o.g. was explained to him

Dijkstra's w.l.o.g. principle

$\varphi, \psi : A^2 \rightarrow \mathbf{Prop}$ predicates on ordered pairs satisfying:

1. $(x, y) : A^2 \vdash \varphi(x, y) = \varphi(y, x)$ and similarly for ψ .
2. finding a third predicate $\xi : A \rightarrow \mathbf{Prop}$ such that $(x, y) : A^2 \vdash \varphi(x, y) \wedge \xi x \implies \psi(x, y)$.
3. showing that $\varphi(x, y) \implies (\xi x \vee \xi y)$;

concluding that, w.l.o.g., $\varphi(x, y) \implies \psi(x, y)$.

Dijkstra's w.l.o.g. principle as a $\text{wlog}(c, p)$

$(x, y) : A^2 \vdash \varphi, \psi : \text{Prop}$ predicates on ordered pairs satisfying:

1. φ, ψ invariant $\implies \mathbf{S}_2$ action on them
3. predicate $\xi : A \rightarrow \text{Prop}$ and proof $q : \varphi(x, y) \implies (\xi x \vee \xi y)$ provide a canoniser:

$$(x, y) : A^2 \vdash c : \varphi(x, y) \rightarrow \underline{\mathbf{S}}_2 \quad cp := \begin{cases} \xi x : & \mathbf{I} \\ \xi y : & \dashv \end{cases} \quad p \in \text{Can } c \iff \varphi(x, y) \wedge \xi x$$

2. finding a core function $(x, y) : A^2 \vdash f : \text{Can } c \rightarrow \psi(x, y)$.

We can now prove: $(x, y) : A^2 \vdash \text{wlog}(c, f) : \varphi(x, y) \implies \psi(x, y)$.

Misery of the unordered pair

Dijkstra recounts: when the w.l.o.g. argument was presented/justified to him informally, the presenter included this step:

4. 'by similar reasoning, $(x, y) : A^2 \vdash \varphi(x, y) \wedge \xi y \implies \psi(x, y)$ '

Dijkstra argues: replacing φ, ψ with **unordered pairs** predicates:

$$\Phi : (A^2 / (x, y) \sim (y, x)) \rightarrow \text{Prop}$$

step (4) coincides with step (2), hence unnecessary.

Arguing what Dijkstra must or may not be forced to concede is in the realm of **Philosophy of Mathematics**—out of scope.

Some observations

- **quotients are subtle**, esp. in the presence of multiple symmetries. Compare:

let S, T be two equilateral triangles in the plane, then S and T may be different

vs

let S, T be two equilateral triangles in the plane, each quotiented by affine symmetries. Then $S = T$.

NB: a standard subtlety when working with quotients.

- '(2) and (4) **coincide**' means '**judgemental** equality of symmetric unordered pairs' McBride's **Ripley phenomena**¹⁰ kicks-in causing **undedicable type-checking**.

NB: may be only a complication due to the dependently-typed metalanguage

¹⁰See other Ripley phenomena in McBride's note: <http://strictlypositive.org/Ripley.pdf>.

Dijkstra's w.l.o.g. misery

Arguing what Dijkstra must or may not be forced to concede is in the realm of **Philosophy of Mathematics**—out of scope.

Some observations

- Dijkstra's misery may be a **strawman**
Whoever presented w.l.o.g. to Dijkstra did **not explicate** the formal argument.

Relevant **symmetry**, predicate **equivariance** and **canoniser** left implicit.

Presenter may have **demonstrated** how the symmetric argument looks like.
Necessary repetition to re-construct implicit data.

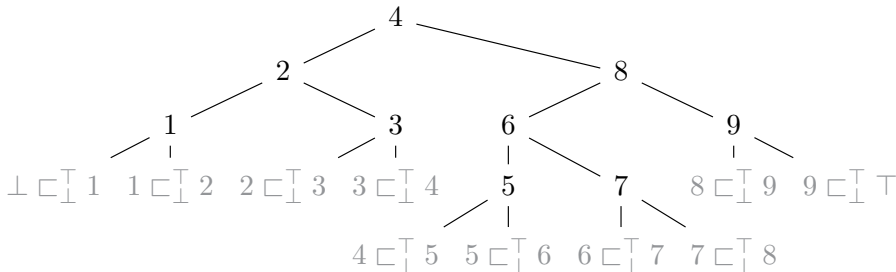
Once symmetry, equivariance, and canoniser are explicit, proving the symmetric case indeed becomes unnecessary, but not before!

Talk structure

- ▶ Introduction
- ▶ Remedial group theory basics
- ▶ Simply-typed symmetric programming
- ▶ Dependently-typed symmetric programming
 - ▶ Indexed actions and w.l.o.g.
 - ▶ Mathematical reasoning: Dijkstra'95 & Harrison'09
- ▶ Symmetric user-defined data-types
 - ▶ Simply-typed data-types
 - ▶ Indexed data-types
- ▶ Not today: Equivariant reasoning
- ▶ Conclusion

McBride's binary search trees [McBride'14]

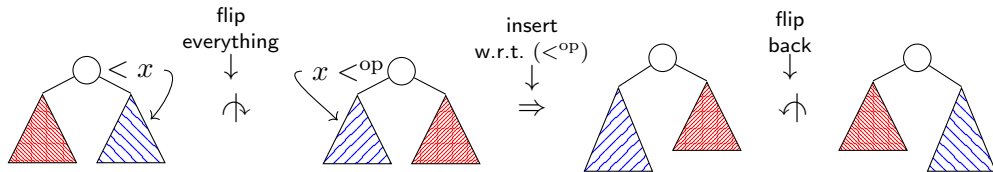
$$\frac{p : a \sqsubset_{\perp}^{\top} b}{\text{Leaf}_p : \mathbb{T}_{(\sqsubset)}(a, b)} \quad \frac{\ell : \mathbb{T}_{(\sqsubset)}(a, x) \quad r : \mathbb{T}_{(\sqsubset)}(x, b)}{\text{Node} \langle x, \ell, r \rangle : \mathbb{T}_{(\sqsubset)}(a, b)}$$



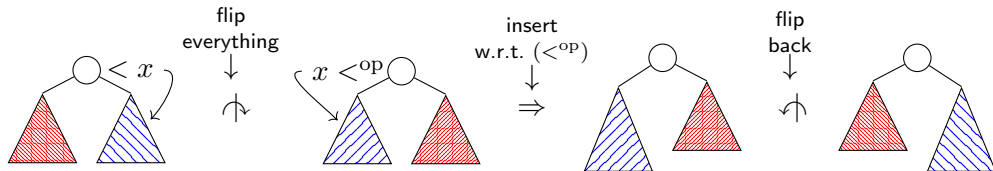
proofs stored in **leafs** keep the **nodes in order**.

our goal: define insertion without the **symmetric case**

$$(\sqsubset) : \mathbf{Total}_{\mathbb{K}}, k : \mathbb{K} \vdash \text{insert} : ((x, y) : \text{interval}_{\mathbb{K}}) \rightarrow x \sqsubset_{\perp}^{\top} k \sqsubset_{\perp}^{\top} y \rightarrow \mathbb{T}_{(\sqsubset)}(x, y) \rightarrow \mathbb{T}_{(\sqsubset)}(x, y)$$



Tree.lean



Tree.lean

Challenges:

- ▶ Symmetry compatible induction principles
- ▶ We'll develop a theory of data-types with symmetry
augments usual theory of data-types with group actions.

Data-types with symmetry

Plan

- ▶ recap: semantic structure for **simply-typed** data-types
- ▶ **simply-typed** data-types with symmetry
- ▶ recap: **indexed** data-types
- ▶ **indexed** data-types with symmetry

Recap: semantic structure for simply-typed data-types

data types	top-level constructor + sub-structure	example structural induction
$\text{List } A = \text{Nil}$ $\quad \{ (::) \} \times A \times \text{List } A$	$X \mapsto \text{Nil}$ $\quad \{ (::) \} \times A \times X$	length: $\begin{cases} \text{Nil} & \mapsto 0 \\ ((::), a, n) & \mapsto 1 + n \end{cases}$
$\text{Tree } A = \text{Leaf}$ $\quad \{ \text{Node} \} \times \text{Tree } A$ $\quad \quad \times A \times \text{Tree } A$	$X \mapsto \text{Leaf}$ $\quad \{ \text{Node} \} \times X$ $\quad \quad \times A \quad \times X$	depth: $\begin{cases} \text{Leaf} \mapsto 0 \\ \langle \text{Node}, m, x, k \rangle \\ \quad \mapsto 1 + \max \{ m, k \} \end{cases}$

etc. for strictly positive data-types

Recap: semantic structure for simply-typed data-types

General picture:

data type + constructors	top-level constructor + sub-structure	structural induction
	functor $F : \mathbf{Set} \rightarrow \mathbf{Set}$ FX : constructors available using substructures $x \in X$	
initial algebra $\mathbf{roll} : F(\mu F) \xrightarrow{\cong} \mu F$ constructs data from constructor + data for substructures evaluates each constructor homomorphically using algebra structure:		algebra $f : FA \rightarrow A$ evaluate constructor given values for sub-structures
	$\frac{f : FA \rightarrow A}{\mathbf{fold} \ f : (\mu F, \mathbf{roll}) \rightarrow (A, f)}$	F -algebra F -algebra homomorphism

Simply-typed data-types with **symmetry**

Examples

- ▶ Pointwise symmetry on lists (G -action A):

$$\boxed{\text{Nil}} \xrightarrow{g \circledast} \boxed{\text{Nil}} \qquad \boxed{a :: x} \xrightarrow{g \circledast} \boxed{(g \circledast a) :: (g \circledast x)}$$

- ▶ Tree flipping symmetry (S_2 -action A):

$$\boxed{\text{Leaf}} \xrightarrow{g \circledast} \boxed{\text{Leaf}} \qquad \begin{array}{ccc} & \boxed{n} & \\ x & / \quad \backslash & y \end{array} \xrightarrow{\text{flip} \circledast} \begin{array}{ccc} & \boxed{n} & \\ \text{flip} \circledast y & / \quad \backslash & \text{flip} \circledast x \end{array}$$

Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

natural transformation

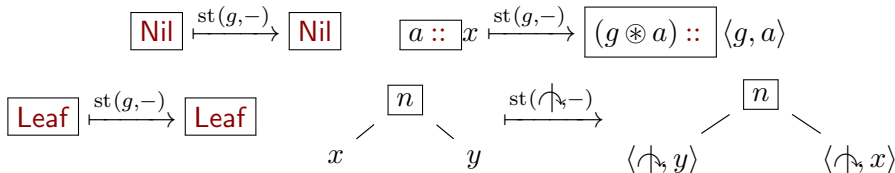
$$\text{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \text{st}_X(e, t) = \text{map}(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \text{st}_X(g * h, t) = \text{map}(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\text{st}_{\underline{G} \times X}(g, \text{st}_X(h, t)))$$

Examples



Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

natural transformation

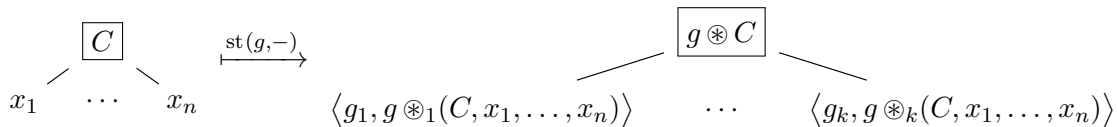
$$\text{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \text{st}_X(e, t) = \text{map}(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \text{st}_X(g * h, t) = \text{map}(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\text{st}_{\underline{G} \times X}(g, \text{st}_X(h, t)))$$

Generally



Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

natural transformation

$$\mathrm{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \mathrm{st}_X(e, t) = \mathrm{map}(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \mathrm{st}_X(g * h, t) = \mathrm{map}(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\mathrm{st}_{\underline{G} \times X}(g, \mathrm{st}_X(h, t)))$$

Unit law

$$\begin{array}{ccc} \begin{array}{c} \boxed{C} \\ \swarrow \quad \searrow \\ x_1 \quad \cdots \quad x_n \end{array} & \xrightarrow{\mathrm{st}(e, -)} & \begin{array}{c} \boxed{C} \\ \swarrow \quad \searrow \\ \langle e, x_1 \rangle \quad \cdots \quad \langle e, x_n \rangle \end{array} \end{array}$$

Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

natural transformation

$$\text{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \text{st}_X(e, t) = \text{map}(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \text{st}_X(g * h, t) = \text{map}(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\text{st}_{\underline{G} \times X}(g, \text{st}_X(h, t)))$$

Multiplication law

$$\begin{array}{c} \boxed{C} \xrightarrow{\text{st}(h, -)} \boxed{h \otimes C} \xrightarrow{\text{map}(\langle (*), - \rangle \circ \text{st}(g, -))} \boxed{(g * h) \otimes C} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \quad \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ x_1 \quad \cdots \quad x_n \langle h_1, y_1 \rangle \quad \cdots \quad \langle h_k, y_k \rangle \langle (g * h)_1, z_1 \rangle \quad \cdots \quad \langle (g * h)_k, z_k \rangle \end{array}$$

Symmetry strengths lift actions

Proposition (lifted action $F_{\text{st}}A = \langle F\underline{A}, (\overset{\text{st}}{*}) \rangle$)

$$\frac{G\text{-action } (*) : \underline{G} \times \underline{A} \rightarrow \underline{A}}{G\text{-action } (\overset{\text{st}}{*}) : \underline{G} \times F\underline{A} \xrightarrow{\text{st}} F(\underline{G} \times \underline{A}) \xrightarrow{F(*)} F\underline{A}}$$

Moreover:

$$\frac{h : A \overset{*}{\rightarrow} B \quad \text{equivariant}}{Fh : F_{\text{st}}A \overset{*}{\rightarrow} F_{\text{st}}B \quad \text{equivariant}}$$

Lifted action

Examples

- Pointwise symmetry on lists (G -action A):

$$\boxed{\text{Nil}} \xrightarrow{g \circledast^{\text{st}}} \boxed{\text{Nil}} \qquad \boxed{a :: x} \xrightarrow{g \circledast^{\text{st}}} \boxed{(g \circledast a) :: (g \circledast x)}$$

- Tree flipping symmetry (S_2 -action A):

$$\boxed{\text{Leaf}} \xrightarrow{g \circledast^{\text{st}}} \boxed{\text{Leaf}} \qquad \begin{array}{c} \boxed{n} \\ \swarrow \quad \searrow \\ x \qquad y \end{array} \xrightarrow{\updownarrow \circledast^{\text{st}}} \begin{array}{c} \boxed{n} \\ \swarrow \quad \searrow \\ \updownarrow \circledast y \quad \updownarrow \circledast x \end{array}$$

Equivariant iterators

Equivariant $\langle F, \text{st} \rangle$ -algebra $\langle \underline{A}, (\otimes), f \rangle$

- ▶ G -action $A = \langle \underline{A}, (\otimes) \rangle$
- ▶ equivariant F -algebra $f : F_{\text{st}} A \otimes \rightarrow A$, i.e., an F -algebra $f : F \underline{A} \rightarrow \underline{A}$ such that

$$f(g \overset{F_{\text{st}} A}{\otimes} t) = g \overset{A}{\otimes} f(t) \quad \begin{array}{ccccc} \underline{G} \times F \underline{A} & \xrightarrow{\text{st}} & F(\underline{G} \times \underline{A}) & \xrightarrow{F(\otimes)} & F \underline{A} \\ \text{id} \times f \downarrow & & = & & \downarrow f \\ \underline{G} \times \underline{A} & \xrightarrow{\quad\quad\quad} & & & \underline{A} \\ & & (\otimes) & & \end{array}$$

Equivariant algebra homomorphism $h : \langle A, f \rangle \otimes \rightarrow \langle B, g \rangle$

simultaneously:

- ▶ an F -homomorphism
- ▶ equivariant map $h : A \otimes \rightarrow B$

Main results

Theorem (parameterised initiality [Bird and Paterson'99])

For every F -algebra $f : FX \rightarrow X$ there is a unique **parameterised homomorphism**:

$$\mathbf{pfold} f : \underline{G} \times \mu F \rightarrow X \quad \text{satisfying} \quad \mathbf{pfold} f \langle g, \mathbf{roll} t \rangle = f(\mathbf{map}(\mathbf{st}(g, t)))$$

Theorem (equivariant induction principle)

Let $\mathbf{roll} : F\mu F \rightarrow \mu F$ be an initial algebra for F with G -strength $\mathbf{st}$. Defining

$$(\circledast) := \mathbf{pfold} \mathbf{roll} : \underline{G} \times \mu F \rightarrow \mu F$$

makes $\langle \mu F, \mathbf{roll} \rangle$ into the **initial** equivariant $\langle F, \mathbf{st} \rangle$ -algebra:

$$\frac{\text{equivariant algebra } f : F_{\mathbf{st}} A \circledast \rightarrow A}{\text{equivariant } F\text{-homomorphism } \mathbf{fold} f : \mu F \circledast \rightarrow A}$$

Main result

Example: Pointwise action on lists

$$g \circledast (x_1 :: \cdots :: x_n :: \text{Nil}) = (g \circledast x_1 :: \cdots :: g \circledast x_n :: \text{Nil})$$

Example: Flipping the tree

$$\mathbf{F} \circledast \left(\begin{array}{c} & & 3 \\ & \swarrow & \searrow \\ & 2 & \\ \swarrow & & \searrow \\ 1 & & 4 \end{array} \right) = \left(\begin{array}{c} 3 & & \\ \swarrow & & \searrow \\ & 2 & \\ \swarrow & & \searrow \\ 4 & & 1 \end{array} \right)$$

Main result

In practice

- ▶ Define G -action on data-type using a strength
- ▶ Use w.l.o.g. reasoning to define **equivariant F -algebra**
removes symmetric cases
- ▶ Appeal to equivariant induction principle to obtain equivariance of traversal

Road-map

More in the draft

- ▶ Extension to indexed data-types
- ▶ More examples:
 - ▶ lookup without [Maybe](#);
 - ▶ insertion to McBride's binary search trees
- ▶ Meta-theoretic results for equivariant reasoning:
 - ▶ representation theorem for equivariant maps
 - ▶ completeness of w.l.o.g. reasoning

Future work

- ▶ more examples
- ▶ mechanisation
- ▶ programming abstractions
- ▶ efficient implementation

Symmetric programming

Exploit symmetries for computation and reasoning—**avoid symmetric cases**.

Mathematical foundations for symmetric programming semantics:

- ▶ **Abstractions** for symmetric programming
- ▶ **Simply-typed** reasoning about equivariance
- ▶ **Dependently-typed** and mathematical reasoning
- ▶ User-defined **data-types** through initial algebra semantics and induction principles
 - ▶ Simply-typed data-types
 - ▶ Dependently-typed/indexed data-types